

UNIVERSITÀ
DEGLI STUDI
DI PADOVA

MyVector e distruttore

Stefano Ghidoni

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE



Agenda

- Creare MyVector ispirato a std::vector
- Distruttore

Richiamo: vector

- Abbiamo già usato gli std::vector
  - Sequenza di oggetti uguali
  - Riferimento agli elementi: []
  - Aggiunta di un elemento: push_back()
  - Numero di elementi: size()
  - Accesso con verifica (se richiesto)
- Altri container utili:
  - std::string
  - std::list
  - std::map
  - …

std::vector

- std::vector ha un comportamento intelligente
  - Cambia dimensione per ospitare gli elementi che inseriamo
  - Conosce la propria dimensione
- Questo comportamento non è supportato in HW – una sovrastruttura

Struttura dinamica del vector

- std::vector è una struttura dinamica
  - Diverso dagli array, che hanno dimensione fissa
    - Dovrei poterne modificare la dimensione
  - Non posso usare una struttura statica come una classe:

```
class vector {
    int size, age0, age1, age2, age3;
};
```

Struttura dinamica del vector

- Per poter realizzare una struttura dinamica abbiamo bisogno di poter creare nuovi vettori le cui dimensioni sono note a tempo di esecuzione
- Esiste una zona di memoria che può essere utilizzata in questo modo: il **free store**
- Esiste uno strumento a **basso livello** che ci permette di utilizzare il free store: **allocazione dinamica della memoria**

std::vector

- Usando il free store gli strumenti a disposizione sono molto meno evoluti:
  - Memoria
  - Indirizzi di memoria
- Il comportamento intelligente di std::vector è **implementato nella libreria standard**!

Implementare vector

- Come possiamo implementare un vector di double con le stesse caratteristiche di std::vector?

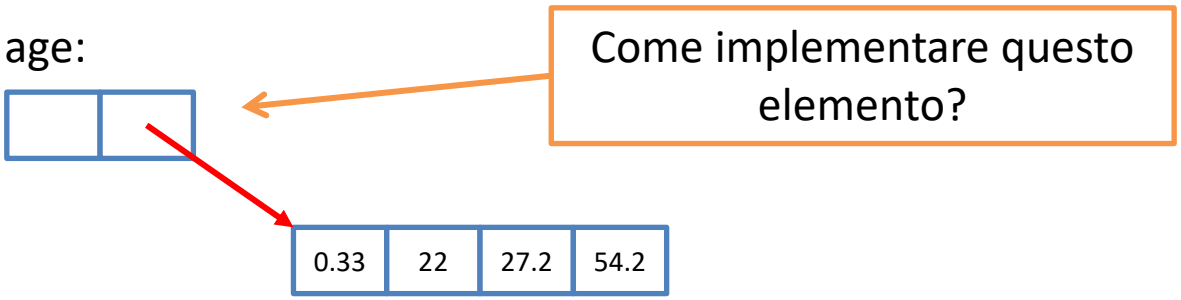
Struttura di un vector

- Consideriamo un uso semplice:

```
vector<double> age(4);

age[0] = 0.33;
age[1] = 22;
age[2] = 27.2;
age[3] = 54.2;
```

- Da realizzare con questa struttura:



MyVector

- Il nostro vector può essere creato così:

```
class MyVector {
  int sz;
  double *elem;
public:
  MyVector(int s);     // crea un array di s double e fa sì
                       // che elem punti ad esso
  int size() const { return sz; }
};
```

- Il puntatore serve per gestire un buffer allocato dinamicamente

- Si consideri questo esempio:

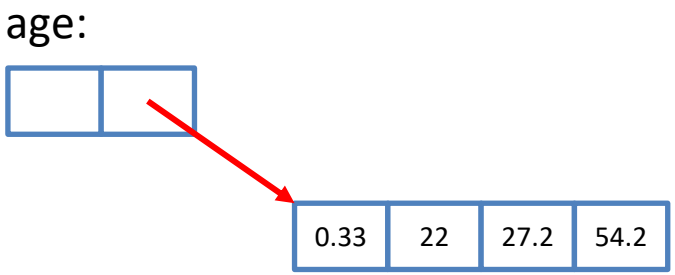
```
void f()
{
    MyVector data(200);
    // utilizzo di data
    return;
}
```

- Che tipo di oggetto è data?
- Cosa succede quando esce dallo scope?

- Anti spoiler ☺

Liberare la memoria con UDT

- La struttura di MyVector è:



- A fine vita, è necessario deallocare la memoria
- Potrei usare una funzione clean_up()
  - Se l'utente si dimentica di chiamarla?
- Come possiamo fare per essere *sicuri* che ciò avvenga in maniera *automatica*?

Distruttore

- Un **distruttore** è una funzione che il compilatore chiama quando un oggetto deve essere distrutto
  - Esce dallo scope
  - Deallocazione di un oggetto
- Il distruttore è chiamato **implicitamente**

Distruttore di MyVector

```
class MyVector{
  int sz;
  double* elem;
public:
  MyVector(int s)
        : sz{ s }, elem{ new double[s]) }
  {
    for (int i = 0; i < s; ++i) elem[i] = 0;
  }

  ~MyVector()
  { delete[] elem; }
  // ...
};
```

Distruttore

Chiamata implicita del distruttore

```
void f3(int n){
  double* p = new double[n];
  MyVector v(n);
  // ... uso p e v ...
  delete[] p;
}
```

v è liberato automaticamente!

- MyVector fa automaticamente ciò che con p dobbiamo fare manualmente

Chiamata implicita del distruttore

- I distruttori sono molto utili quando dobbiamo rilasciare risorse acquisite
  - File
  - Thread
  - Lock
  - Stream
  - …

Distruttori creati dal compilatore

- Quando il distruttore non è implementato esplicitamente, è generato automaticamente dal compilatore (**compiler-generated destructor**)
  - Non è "intelligente": non dealloca al posto nostro
  - Chiama i distruttori di eventuali oggetti membro

Distruttori creati dal compilatore

```
struct Customer {
  std::string name;
  std::vector<string> address;
  // ...
};

void some_fct() {
  Customer Fred;
  // init Fred
  // use Fred
}
```

Qui Fred è distrutto:

- Chiamata a distruttore di string
- Chiamata a distruttore di vector<string>

Pattern acquisizione risorse

- Un oggetto acquisisce risorse nel costruttore
- Durante la sua vita, l'oggetto può:
  - Acquisire altre risorse
  - Liberare alcune risorse
- A fine vita, l'oggetto rilascia tutte le risorse

Azioni di new e delete

- Operatore new
  - Alloca memoria
  - Invoca costruttore
- Operatore delete
  - Invoca distruttore
  - Libera memoria



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

MyVector e distruttore

Stefano Ghidoni

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE